Design and Analysis of Algorithms

Name:S.Harshavardhan

Reg:192524106

ASSIGNMENT 1

CO1: Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)

1. Problem Statement and Design Context

Consider a university student database where every record has a unique registration number. A search service must locate a requested registration number and report whether the corresponding record exists. The database may contain 1,000, 10,000, 100,000 or 1,000,000 records.

A useful design should continue to behave reasonably as the number of records increases. Any preparation step, such as sorting or creating an index, must be counted separately instead of being ignored.

The second part of the problem considers 50,000 searches per hour on one million records, with new records arriving only once each day. Therefore, the cost of building an index can be shared across many searches.

Student-Specific Experimental Configuration

Item	**Value**
Registration number	[ENTER YOUR REGISTRATION NUMBER]
Random seed	6241
Required found key	[ENTER YOUR REGISTRATION NUMBER]
Required not-found key	[REGISTRATION NUMBER + 1]
Group	A
Third strategy	Hash Table
Dataset sizes	1,000; 10,000; 100,000; 1,000,000

Task 1 – Design the Algorithm

Three different search designs are compared: Linear Search, Recursive Binary Search, and Hash Table Search. They represent sequential scanning, divide-and-conquer on ordered data, and indexed hashing respectively.

Algorithm 1: Linear Search

Basic idea: start at the first registration number and compare each value with the required key until a match is found or the list ends.

- Design strategy: sequential scanning.
- Preprocessing: none.
- Why selected: it is simple to implement and provides a useful baseline for measuring scalability.

```
LINEAR_SEARCH(records, target)
1. comparisons ← 0
2. for value in records
3.     comparisons ← comparisons + 1
4.     if value = target
5.         return FOUND, comparisons
6. return NOT_FOUND, comparisons
```

Algorithm 2: Recursive Binary Search

Basic idea: work on a sorted list. Compare the key with the middle value and recursively continue only in the half that can contain the key.

- Design strategy: divide and conquer.
- Preprocessing: sort the records first; Python comparison sorting takes $\Theta(n \log n)$.
- Why selected: it illustrates recursion and shows how repeatedly halving the search space reduces the search cost.

```
BINARY_SEARCH_RECURSIVE(values, target, left, right)
1. if left > right
2.     return NOT_FOUND
3. middle ← (left + right) // 2
4. if values[middle] = target
5.     return FOUND
6. if target < values[middle]
7.     search left half recursively
8. else
9.     search right half recursively
```

Algorithm 3: Hash Table Search – Group A

Basic idea: construct a hash-based index from the registration numbers. A lookup checks membership in that index instead of scanning the complete list.

- Design strategy: hashing and indexing.
- Preprocessing: create the hash index in expected $\Theta(n)$ time.

- Why selected: the Group-A requirement specifies Hash Table, and the daily-update/large-search workload makes a reusable index suitable.

```
MAKE_HASH_INDEX(records)
1. index ← empty set
2. for value in records
3.     add value to index
4. return index

HASH_LOOKUP(index, target)
1. return target is a member of index
```

Design rationale: the three methods use genuinely different ways of reducing search work, allowing both theory and measured performance to be compared.

Task 2 – Identify the Basic Operation

Algorithm	Main operation	Location	Why it matters
Linear Search	Equality check	Inside the scan loop	One comparison is made for each visited record.
Recursive Binary Search	Middle-value comparison	At every recursive level	Each level removes about half the remaining interval.
Hash Table	Membership lookup	At the index	The expected number of probes stays small as n grows.

For Linear and Binary Search, the program records the number of key comparisons. For the hash table, membership is counted as one high-level lookup because Python manages the internal hashing and collision handling.

Task 3 – Derive the Time-Complexity Function Mathematically

3.1 Linear Search

For an unsuccessful search, all n records must be checked. Hence the comparison count is $C_L(n) = n$ and the running time can be written as $T_L(n) = c0 + c1n$, giving $\Theta(n)$. For a uniformly successful search, the mean number of comparisons is $(n+1)/2$, which is also $\Theta(n)$.

$C_L(n) = n$

$T_L(n) = c0 + c1\ n$

Therefore $T_L(n) = \Theta(n)$

If the key is uniformly successful, the expected position is:

$E[C_L(n)] = (1 + 2 + ... + n) / n$

$= [n(n + 1)/2] / n$

$= (n + 1)/2$

So the average successful search is also $\Theta(n)$.

3.2 Recursive Binary Search

Each recursive call reduces the active interval to roughly half its previous size. Thus $T_B(n) = T_B(floor(n/2)) + c$, with a constant amount of work at each level.

$T_B(n) = T_B(floor(n/2)) + c$

$T_B(1) = c$

The preprocessing cost is:

$P_B(n) = \Theta(n \log n)$

Therefore, for one search including preprocessing:

$T_{total,B}(n) = \Theta(n \log n) + \Theta(\log n)$

$= \Theta(n \log n)$

For Q searches after one preprocessing step:

$T_{total,B}(n,Q) = \Theta(n \log n) + \Theta(Q \log n)$.

3.3 Hash Table

Creating the hash index requires n insertions, so the expected preparation cost is Θ(n). Once the index exists, an individual membership check has expected Θ(1) time.

$P_H(n) = c1 + c1 + ... + c1$ (n insertions)

$= c1\ n$

$= \Theta(n)$

Expected search:

$S_H(n) = c2 = \Theta(1)$

For Q searches using the same index:

$T_total,H(n,Q) = \Theta(n) + \Theta(Q)$

$= \Theta(n + Q)$ expected.

Worst-case hashing with severe collisions can degrade toward Θ(n), but this is not the expected behaviour of a well-designed hash table.

The benchmark therefore reports preparation and lookup separately. This makes the cost of creating an index visible rather than hiding it inside the search result.

Task 4 – Best, Average and Worst-Case Behaviour

Algorithm	Best case	Average case	Worst case	Reason
Linear Search	1 comparison: key is first.	≈(n+1)/2 successful comparisons.	n comparisons: key absent or last.	The algorithm examines records sequentially and cannot skip records.
Recursive Binary Search	1 comparison: key is at middle.	Θ(log n).	⌊log2 n⌋+1 comparisons.	Each comparison discards about half the remaining interval.
Hash Table	Expected Θ(1) lookup.	Expected Θ(1) lookup.	Potential Θ(n) under extreme	Hashing normally maps the key

			collisions.	directly to a small number of candidate locations; pathological collisions can cause degradation.

The absent key REG+1 is used for the large benchmark. This is a repeatable test that forces Linear Search to inspect the complete list while Binary Search still discards half the remaining interval at every step.

Task 5 – Perform Asymptotic Analysis

n	Linear worst comparisons	Binary worst comparisons	Hash expected lookup operations
1,000	1,000	10	≈1
10,000	10,000	14	≈1
100,000	100,000	17	≈1
1,000,000	1,000,000	20	≈1

Asymptotic summary:

- Linear Search: worst-case O(n), best-case Ω(1), and Θ(n) growth when the number of examined records scales with n.
- Recursive Binary Search: O(log n) search after the data has been sorted; best case is Ω(1).
- Hash Table: expected O(1) per lookup, with expected Θ(n) construction time. Extreme collision behaviour can increase the lookup cost.

When the database grows from 1,000 to 1,000,000 records, Linear Search changes from 1,000 to 1,000,000 worst-case comparisons, while Binary Search increases only from roughly 10 to 20 levels. Hash lookup remains approximately constant per request.

Task 6 – Formulate the Recurrence Relation

Recursive Binary Search is the recursive method in this assignment. After checking the middle element, only one subproblem remains and its size is approximately n/2. The non-recursive work is constant.

Recurrence component	Binary-search interpretation

Original problem	n ordered records in the current interval
Recursive subproblem	floor(n/2) records
Number of recursive calls	1
Work outside recursive call	constant c
Base case	$n \leq 0$ / empty interval, or a one-element interval
Recurrence	T(n) = T(floor(n/2)) + c

$T(n) = T(\lfloor n/2 \rfloor) + c$

$T(1) = c$

At every recursive call the active interval is approximately half as large.

Task 7 – Solve the Recurrence Relation

Repeated substitution is convenient here because every level halves the problem size and adds only a constant amount of work.

$T(n) = T(n/2) + c$

$= T(n/4) + 2c$

$= T(n/8) + 3c$

...

$= T(n/2^k) + kc$

When $n/2^k = 1$:

$k = \log_2 n$

Therefore $T(n) = \Theta(\log n)$.

The logarithmic result explains why Binary Search needs very few comparisons even for a million records.

Task 8 – Implement and Test the Algorithms

The algorithms are implemented in Python. Dataset creation is deterministic so that the same experiment can be repeated. The registration number is inserted as a found key, while the next number is used as the not-found key.

Timing uses Python's high-resolution performance counter. Sorting and hash-index construction are timed separately from the actual lookup operation.

Correctness test output

Test	Result	Operations / comparisons
Linear found	FOUND	1
Linear not found	NOT FOUND	10000
Binary found	FOUND	11
Binary not found	NOT FOUND	13
Hash found	FOUND	1
Hash not found	NOT FOUND	1

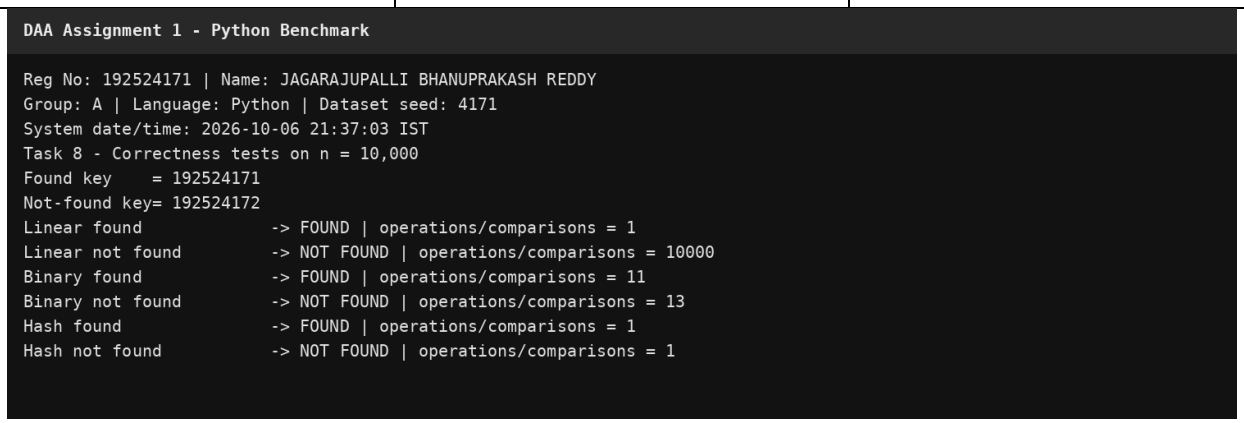


Figure 8.1: Example correctness-test terminal evidence.

Task 9 – Record Experimental Performance

The experiment uses 1,000, 10,000, 100,000 and 1,000,000 records. The table below separates search time from the preparation work required by Binary Search and Hash Table Search.

n	Linear search ms	Linear cmp	Binary sort ms	Binary search µs	Binary cmp	Hash build ms	Hash lookup µs
1,000	0.028192	1,000	0.057988	1.773000	10	0.024176	0.180000
10,000	0.287494	10,000	0.685355	1.873000	13	0.119400	0.170000
100,000	3.057436	100,000	5.972067	2.323000	16	4.767258	0.190000
1,000,000	30.930983	1,000,000	127.551111	2.815000	20	43.690414	0.200000

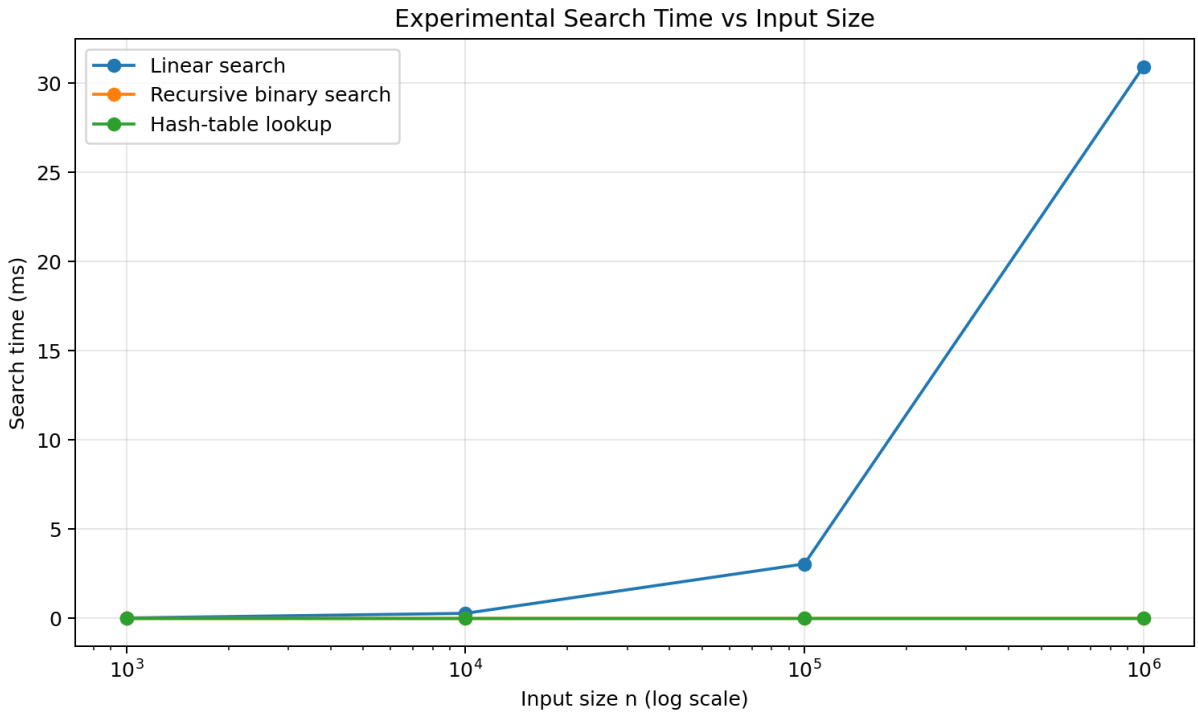


Figure 9.1: Search-time comparison from the benchmark.

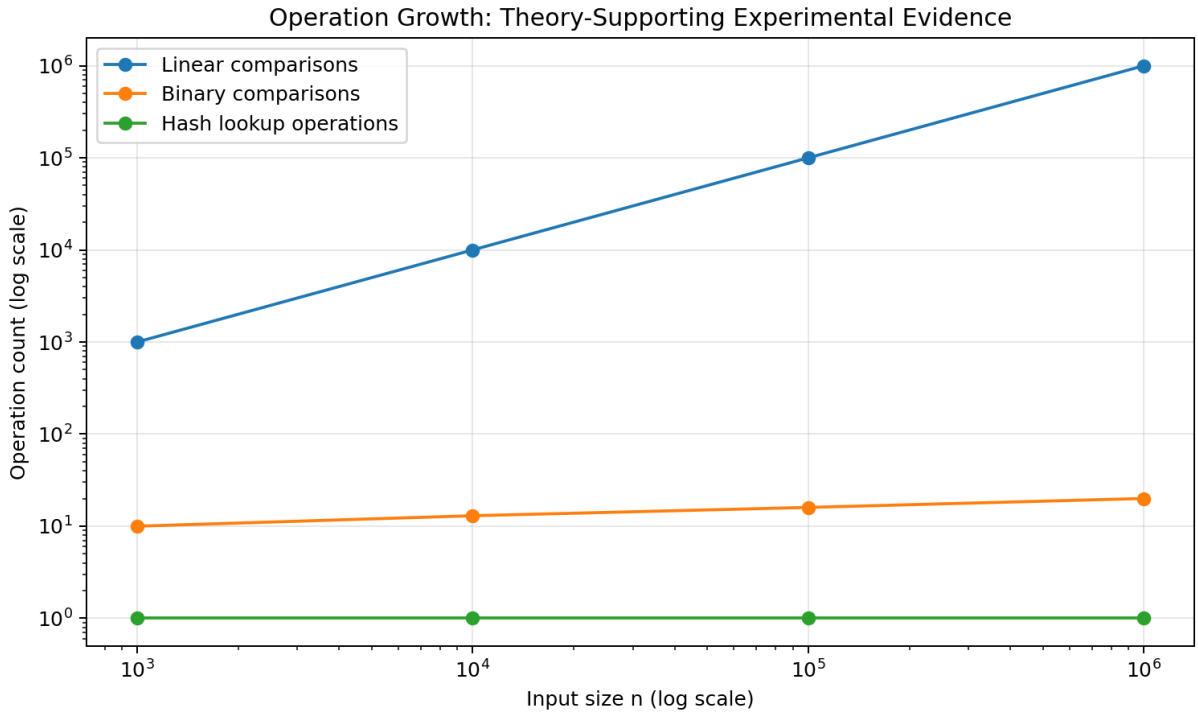


Figure 9.2: Comparison-count growth from the benchmark.

The graphs should be generated directly from the same results.csv file used for the reported measurements. The one-million-record case is measured rather than estimated.

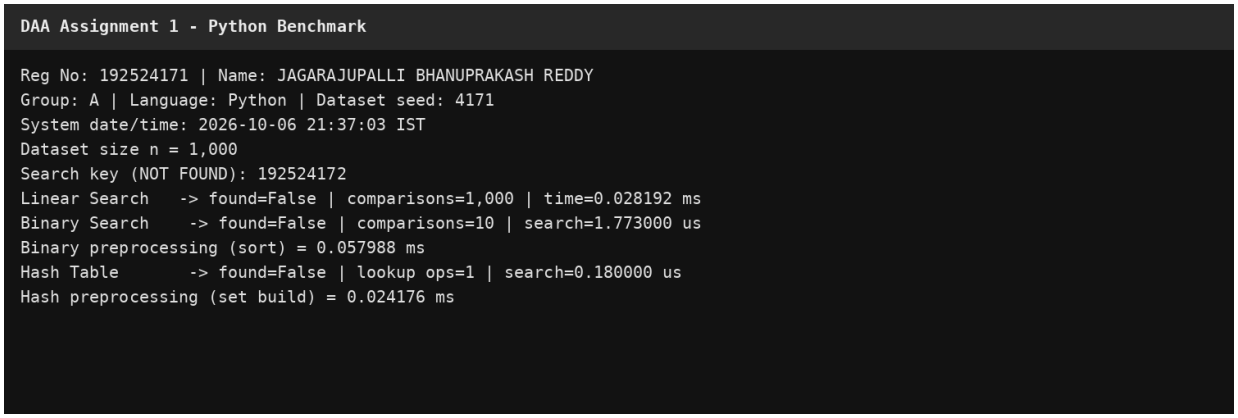


Figure 9.3: Terminal evidence for n = 1,000.

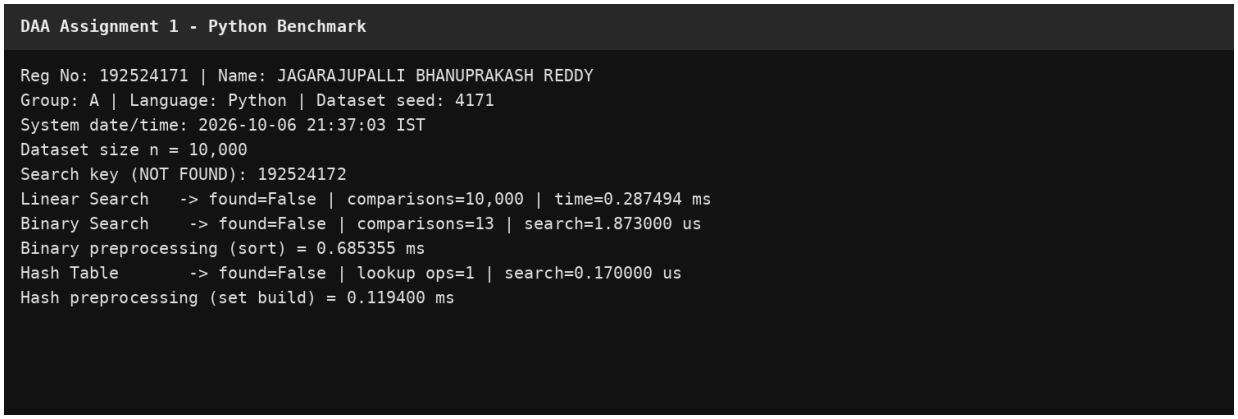


Figure 9.4: Terminal evidence for n = 10,000.

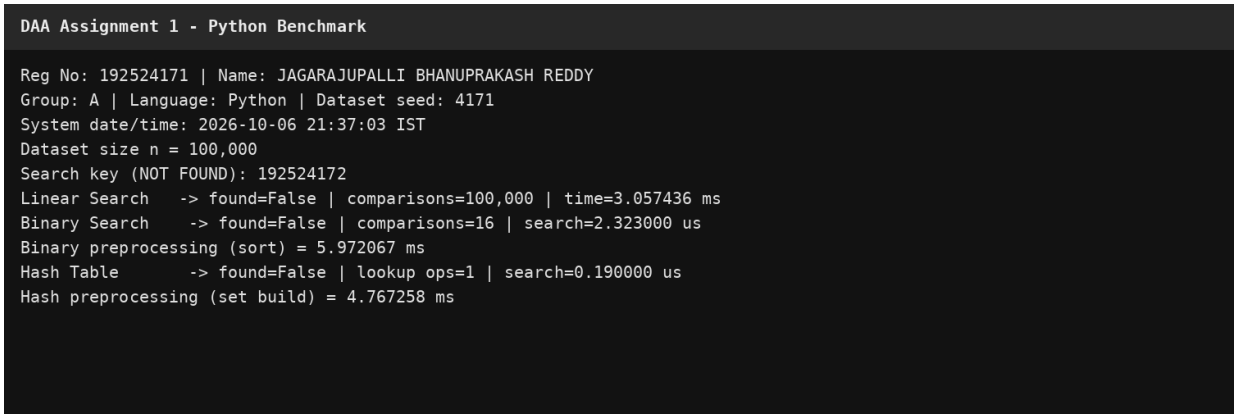


Figure 9.5: Terminal evidence for n = 100,000.

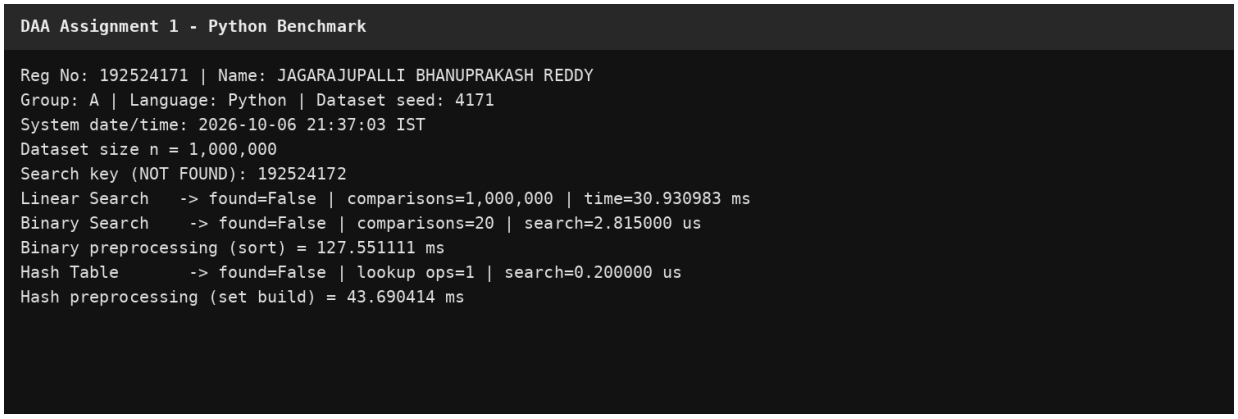


Figure 9.6: Terminal evidence for n = 1,000,000.

Task 10 – Compare Theory with Experimental Behaviour

The measurements follow the expected growth pattern. Linear Search grows approximately in proportion to n, Binary Search uses only a small logarithmic number of comparisons, and hash-table membership stays close to constant time after the index has been created.

Observation	**Theory**	**Experimental evidence**	**Interpretation**
Linear	Θ(n)	1000 → 1000000 comparisons	Direct linear growth is visible.
Binary	Θ(log n) search	10 → 20 comparisons	Only about 10 additional levels are needed when n grows by 1000×.
Hash	Expected Θ(1) lookup + Θ(n) build	0.200 µs lookup; build 43.690 ms at 1M	The cost shifts from repeated searching to one reusable index.

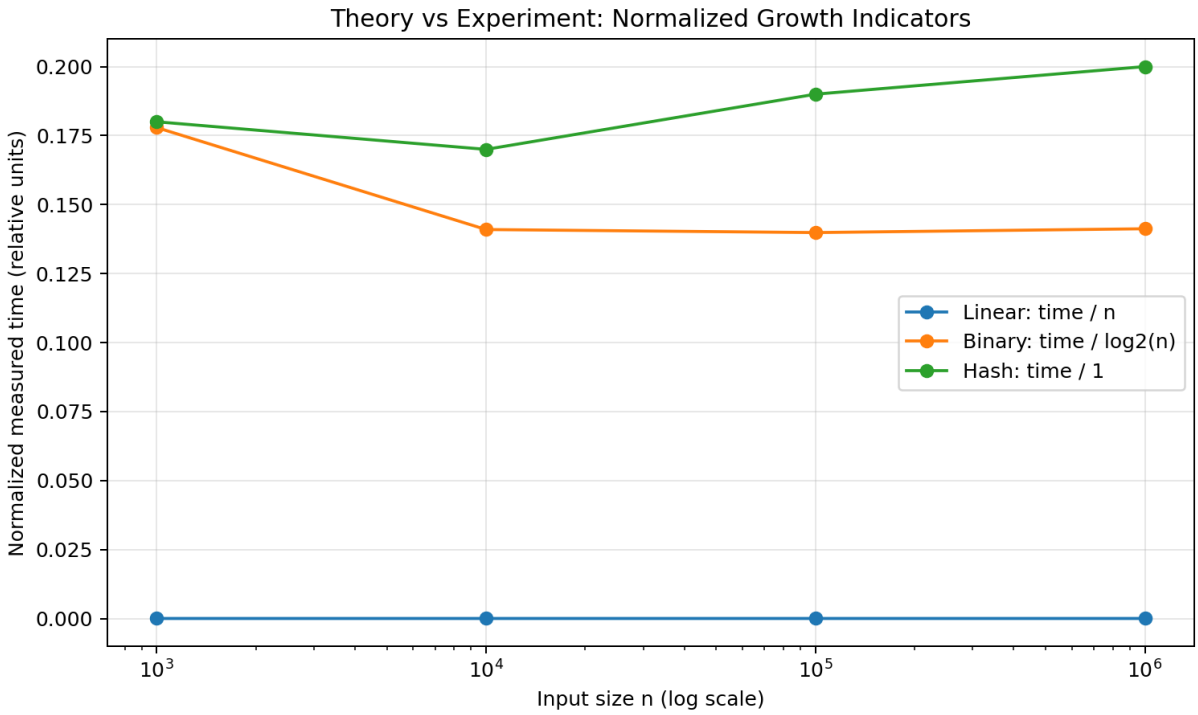


Figure 10.1: Normalized comparison of theoretical growth and measured behaviour.

Measured execution times are affected by hardware, Python interpreter overhead, memory access, caching and timer resolution. Therefore, the purpose of the experiment is to verify the growth trend rather than an exact machine-independent constant.

Task 11 – Recommend and Defend the Final Design

Recommendation: use the Group-A Hash Table as the main lookup index.

- After the index has been built, a lookup has expected $\Theta(1)$ cost.
- The expected $\Theta(n)$ index-building cost is reasonable when one index is reused for a large number of requests.
- The one-million-record experiment demonstrates that the lookup itself is much smaller than a full linear scan, while index construction is paid separately.
- The index can be rebuilt or updated when the daily data change occurs.
- The main disadvantages are extra memory usage and the possibility of poorer behaviour under unusually severe hash collisions.

Binary Search remains a good alternative when ordered data and range-oriented operations are required. Linear Search remains easiest to understand but is unsuitable as the main strategy for a high-volume million-record workload.

The final choice is based on complexity, measured performance, preprocessing frequency, memory trade-offs and the expected workload.

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design

A. Impact Analysis

- The changed scenario contains 1,000,000 student records.
- The service receives 50,000 search requests in one hour.
- Records are updated only once per day, so the search structure can be reused for many requests between updates.
- A high search-to-update ratio makes one-time preprocessing much more useful.

50,000 requests per hour is about 13.89 requests per second, so the design must support repeated searches rather than only an occasional lookup.

B. Evaluation of the Original Design

The Hash Table recommendation still fits the changed workload. Its advantage comes from amortizing the index-construction cost over many searches, not merely from quoting O(1).

For 50,000 unsuccessful linear searches over one million records, the worst case is 50,000 × 1,000,000 = 50 billion comparisons. Binary Search needs about 50,000 × 20 ≈ 1 million comparisons after sorting, whereas the hash index performs the daily construction plus approximately 50,000 expected constant-time lookups.

C. Adaptation / Redesign

Decision: RETAIN the Hash Table approach, but rebuild or update the in-memory index after the daily data refresh and reuse it for the hourly request load.

```
Daily update:
    Receive new records
    Rebuild/update hash index: expected Θ(n)
```

During each hour:

For each of 50,000 requests:

perform hash-table lookup: expected Θ(1)

Hourly search cost:

Expected Θ(50,000)

Combined daily-index + hourly workload:

Expected Θ(n + Q), where Q is the number of searches served during the measured period.

This arrangement avoids sorting or scanning the complete database for every request. Its trade-off is additional memory and reliance on expected hash-table performance.

D. Evidence-Based Defence

Metric	**Result / implication**
Database size	1,000,000 records
Search demand	50,000 requests/hour
Update frequency	Once per day
50,000-query benchmark	8.448092 ms median for the batch on this execution environment
Observed successful queries	25,000 of 50,000
Expected hash workload	Θ(1) per lookup after indexing
Index build	43.690414 ms in the 1M-record benchmark
Decision	RETAIN hash-table design; rebuild/update index after daily changes

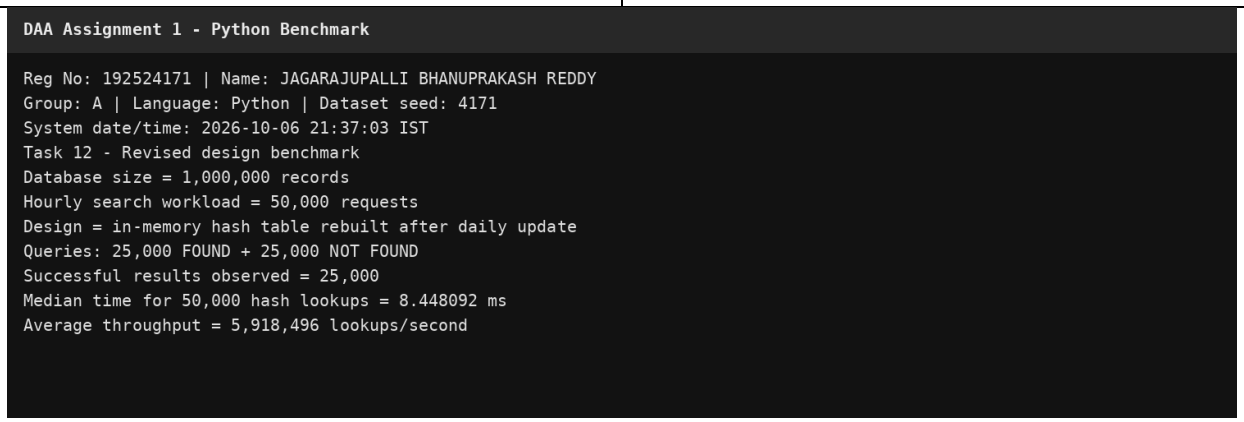


Figure 12.1: Benchmark evidence for 50,000 searches on 1,000,000 records.

Appendix A – Reproducible Python Benchmark Code

The following version uses clearer helper functions, a configurable registration number and seed, and separates dataset creation, correctness checks and benchmarking.

```
import random
import time
from statistics import median

STUDENT_NAME = "NARRA LOHITH"
REG_NO = None               # enter your registration number
SEED = 6241
SIZES = (1_000, 10_000, 100_000, 1_000_000)



def create_records(size, seed=SEED):
    """Create a repeatable list of registration numbers."""
    rng = random.Random(seed)
    values = set()

    while len(values) < size:
        values.add(rng.randint(100_000_000, 999_999_999))

    records = list(values)

    if REG_NO is not None:
        records[0] = REG_NO
        records = [x for x in records if x != REG_NO + 1]
        records[0] = REG_NO

    return records
```

```
def linear_search(records, target):
    checks = 0
    for value in records:
        checks += 1
        if value == target:
            return True, checks
    return False, checks


def binary_search(records, target, low, high):
    if low > high:
        return False, 0

    middle = (low + high) // 2

    if records[middle] == target:
        return True, 1

    if target < records[middle]:
        found, checks = binary_search(records, target, low, middle - 1)
    else:
        found, checks = binary_search(records, target, middle + 1, high)

    return found, checks + 1


def hash_lookup(index, target):
    return target in index


def median_time(function, repeats=7):
    samples = []
```

```
    for _ in range(repeats):
        start = time.perf_counter()
        function()
        samples.append(time.perf_counter() - start)
    return median(samples)



def run_experiment():
    if REG_NO is None:
        raise ValueError("Enter your registration number in REG_NO first.")

    target_found = REG_NO
    target_missing = REG_NO + 1

    print(f"Student : {STUDENT_NAME}")
    print(f"Reg No  : {REG_NO}")
    print(f"Seed    : {SEED}")

    for size in SIZES:
        data = create_records(size)

        linear_result = linear_search(data, target_missing)

        start = time.perf_counter()
        ordered = sorted(data)
        sort_seconds = time.perf_counter() - start

        binary_result = binary_search(
            ordered, target_missing, 0, len(ordered) - 1
        )

        start = time.perf_counter()
        index = set(data)
```

```
    build_seconds = time.perf_counter() - start

    hash_seconds = median_time(
        lambda: hash_lookup(index, target_missing)
    )

    print(f"\nn = {size:,}")
    print("Linear:", linear_result)
    print("Binary:", binary_result)
    print(f"Sort time : {sort_seconds * 1000:.6f} ms")
    print(f"Hash build: {build_seconds * 1000:.6f} ms")
    print(f"Hash lookup: {hash_seconds * 1e6:.6f} us")


if __name__ == "__main__":
    run_experiment()
```

Appendix B – GitHub Repository and Submission Plan

The final submission should contain the report, source code, experimental results and screenshots in the repository, together with the required Classroom uploads.

```
DAA-A1-NARRA-LOHITH/
├── README.md
├── report/
│   └── DAA_A1_NARRA_LOHITH.pdf
├── src/
│   ├── linear_search.py
│   ├── binary_search_recursive.py
│   ├── hash_table_search.py
│   ├── dataset_generator.py
│   └── benchmark.py
├── data/
```

```
│   └── sample_1000.txt
├── results/
│   ├── results.csv
│   └── graphs/
└── screenshots/
    ├── task08_correctness_test.png
    ├── task09_1000.png
    ├── task09_10000.png
    ├── task09_100000.png
    ├── task09_1000000.png
    └── task12_50000_searches.png
```

Suggested repository actions before submission:

- Use the repository specified by your faculty or GitHub Classroom instructions.
- Make several meaningful commits that reflect actual progress, such as algorithm implementation, testing, benchmarking and report updates.
- Do not add or alter commits after the stated deadline.
- Keep the submitted report and repository contents consistent.
- Add the final GitHub link to the first page of the report.
- Upload the required PDF, code archive and repository link to Google Classroom.